Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
Last login: Fri Aug 14 14:25:33 on ttys000
[varnikakomali@VARNIKAs-MacBook-Air ~ % cd OS_LAB
[varnikakomali@VARNIKAs-MacBook-Air OS_LAB % cd Practical
[varnikakomali@VARNIKAs-MacBook-Air Practical % nano practical4.c
[varnikakomali@VARNIKAs-MacBook-Air Practical % gcc practical4.c -o practical4
[varnikakomali@VARNIKAs-MacBook-Air Practical % ./practical4
[Child 1] PID: 13631 sleeping for 1s (Exits early to become Zombie)

[Parent] Created Child 1 (PID: 13631) and Child 2 (PID: 13632)
[Parent] Sleeping 2s to allow Child 1 to terminate and enter Zombie state...
[Child 2] PID: 13632 sleeping for 4s (Longer task)
[Child 1] Terminating now...

=== Investigating Process Table for Zombie State ===
  PID  PPID STAT COMMAND
13631 13630 Z+   <defunct>
13632 13630 S+   ./practical4

=== Synchronizing with waitpid() specifically for Child 2 ===
[Child 2] Terminating now...
[Parent] Reaped Child 2 (PID: 13632) with exit status 20

=== Synchronizing with wait() for Child 1 (Eliminating Zombie) ===
[Parent] Reaped Child 1 (PID: 13631) with exit status 10

=== Final Process Table Check (All Zombies Cleared) ===
  PID  PPID STAT COMMAND
varnikakomali@VARNIKAs-MacBook-Air Practical %
```